

计算机组成原理

实验1 RISC-V汇编程序设计

2026 · 春

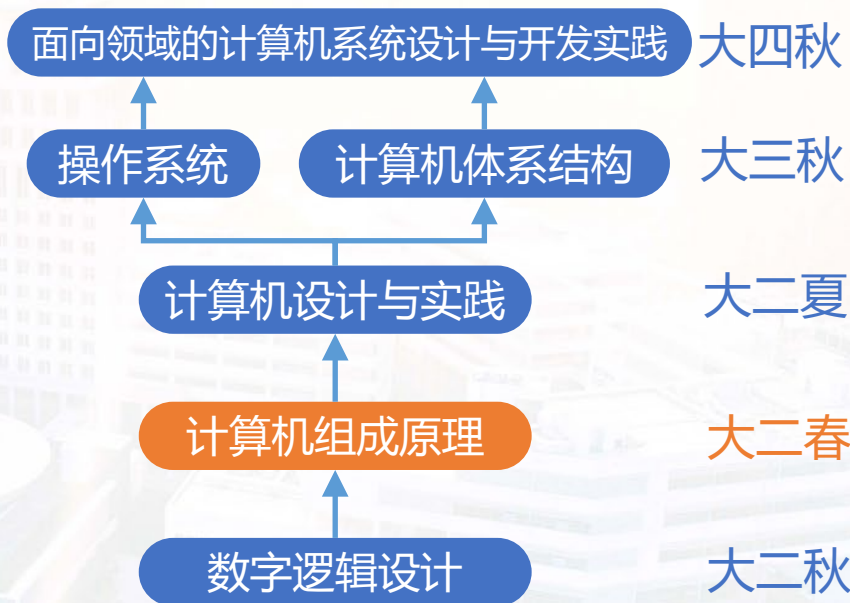
哈工大



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验课程介绍

- 理解计算机组成原理
- 掌握CPU主要部件的设计方法
- 计算机体系结构方向的基础
- 在线指导书：
 - ✓ <https://organ.p.cs-lab.top/>



实验课程介绍

- 实验学时：12学时
- 实验总分：30分

	实验项目	学时	分数	实验平台
实验1	RISC-V汇编程序设计	4	9	RV32 GNU、RARS
实验2	运算器设计	4	12	Vivado 2023.2
实验3	高速缓存器设计	4	12	Vivado 2023.2

实验目的

- 了解C语言[从编译到汇编](#)的过程，了解C程序和汇编语言的对应关系
- 熟悉和理解[子程序工作流程](#)，掌握RISC-V基本[汇编程序的编写](#)
- 掌握RISC-V汇编工具RARS的使用方法
- 了解内联汇编和向量指令



实验内容

- 1、用C语言编写程序，实现IEEE754加法（程序已提供）
- 2、在RV32汇编环境中，完成C程序的预编译、编译、汇编、链接及执行的过程



实验内容

- 3、编写RV32汇编程序，实现矩阵与列向量的乘法(GEMV)，要求：
 - 数据均存放在数据段，均为有符号数

```
1|.data
2|    Result_C: .space 16          # result vector (4 x 32bit)
3|    Matrix_A: .byte 1, 2, 3, 4, 5, 6, 7, 8      # matrix A (4 x 8)
4|              .byte 2, 3, 4, 5, 6, 7, 8, 9
5|              .byte 3, 4, 5, 6, 7, 8, 9, 10
6|              .byte 4, 5, 6, 7, 8, 9, 10, 11
7|    Vector_B: .byte 1, 1, 1, 1, 1, 1, 1, 1      # Vector B (8 x 1)
8|
9|.text
10|    .....
```

- 正确使用子程序、不可使用伪指令

实验原理

◆ 编译 【高级语言(.c) -> 汇编语言(.s/.asm)】

分两步：预编译/预处理 + 编译

A. 预编译 (gcc.gnu.org/onlinedocs/cpp/)

- 去除注释 —— 单行注释//、多行注释/* */
- 处理预编译指令
 - 包含文件 —— 将被包含的文件插入到相应的#include语句处
 - 替换宏定义 —— #define常量替换、表达式代入
 - 处理条件预编译指令 —— #if、#ifdef、#elif、#else、#endif

B. 编译 ("[Compilers: Principles, Techniques, & Tools](#)")

对预处理后的源文件进行词法分析、语法分析、语义分析、中间代码生成、代码优化以及目标代码生成，得到汇编代码



实验原理

◆ 汇编 【汇编语言(.s/.asm) -> 目标文件/对象文件(.o)】

将汇编代码翻译成CPU能识别的机器码，并按照ELF文件的格式标准，将机器码存储在目标文件中

- ELF (Executable & Linkable Format)：一种目标文件/链接库文件/可执行文件的标准文件格式 (flint.cs.yale.edu/cs422/doc/ELF_Format.pdf)
- 目标文件中存储的是机器码 (Machine Language)

每一个汇编语句几乎都对应一条CPU指令。因此，汇编比编译简单，汇编器只需根据汇编语句和CPU指令的对照表——翻译即可

- “几乎”——伪指令等价于多条指令，如RISC-V的li指令可能等价于lui和addi)
- 汇编语句与CPU指令的对照表：ISA手册 (riscv.org/technical/specifications/)



实验原理

◆ 链接 【目标文件(.o) -> 可执行文件】

根据重定向表，将各个目标文件及库文件链接在一起，成为可执行文件

- Link: 把各.o文件中的代码段 (text segment)、数据段 (data segment) 等全部“拼”在一起
- 重定向表由编译器生成，存放在含有主函数的目标文件中，记录了哪些符号以何种方式 (绝对地址/相对地址) 重定位到哪里
 - 查看重定向表的命令: `gcc -r main.o`
 - Ref: mp.ofweek.com/ee/a656714328207
- 库文件的链接方式: 静态链接、动态链接
 - 静态链接: 把目标文件和库文件直接链接在一起
 - 动态链接: 把目标文件和库文件的描述信息链接起来，运行时加载库代码



实验原理

◆ 加载执行

一般由OS加载可执行文件

- 在内存中创建代码段、数据段、堆栈段地址空间
- 拷贝用户参数，初始化寄存器
- 跳转到用户程序，设置PC

◆ 反汇编 【机器码 -> 汇编程序】

反汇编是用于调试和定位处理器问题时最常用的手段之一

- ELF文件并非文本文件，无法用编辑器打开并查看代码
- 若想查看ELF文件中的指令和数据，需要借助反汇编



实验环境

虚拟机：WSL2

操作系统：Debian 12

编译、汇编和链接：RISC-V 32bit GNU

RISC-V模拟器：Spike仿真器 + riscv-pk代理内核（执行RISC-V程序）

自行构建方案：

organ.p.cs-lab.top/lab1/A-envir/



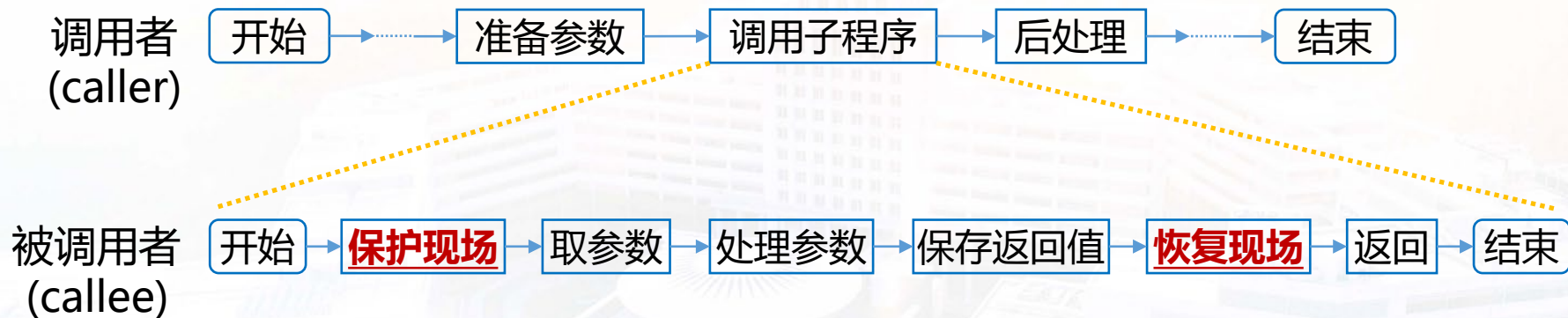
实验演示

以矩阵-向量乘法的C程序为例
演示编译、汇编等过程



实验原理

◆ 子程序执行流程：



实验原理

◆ 子程序框架：

```
14  .text
15  MAIN:
16      .....
17      jal  ra, SOME_FUNC      # Call a sub-routine.
18      .....                  # Here is where the sub-routine returns.
19      ori  a7, zero, 10       # Set system call number(10 for termination).
20      ecall                   # This program terminates here.
21
22  SOME_FUNC:
23      push s0
24      push s1
25      .....                  # Assume that s0 and s1 are modified here.
26      pop  s1
27      pop  s0
28      jalr zero, 0(ra)        # The sub-routine returns.
```

- 关键点：调用、返回、保护现场、恢复现场

实验原理

◆ 保护现场:

- RISC-V规定sp、s0~s11需要callee保护, a0~a7、t0~t6需caller保护
- 若子程序内部修改sp、s0~s11的值, 则需在保护现场时备份寄存器
- 方法: 使用push、pop宏定义保护现场、恢复现场

```
.macro push %a
    addi sp, sp, -4
    sw   %a, 0(sp)
.end_macro

.macro pop %a
    lw   %a, 0(sp)
    addi sp, sp, 4
.end_macro
```

```
SOME_FUNC:
    push t0
    push t1
    .....
    pop  t1
    pop  t0
    jalr zero, 0(ra)
```


实验原理

◆ 伪指令区分：

- 查指令手册
- 借助RARS判断：
 - “Basic” 和 “Source” 下显示的指令不同则是伪指令

Text Segment				
Bkpt	Address	Code	Basic	Source
☐	0x00000000	0x000022b7	lui x5, 2	16: lui t0, 0x00002
☐	0x00000004	0x000022b7	lui x5, 2	17: li t0, 0x00002000
☐	0x00000008	0x00028293	addi x5, x5, 0	
☐	0x0000000c	0x0002a503	lw x10, 0(x5)	18: lw a0, 0(t0)
☐	0x00000010	0x010000ef	jal x1, 0x00000010	19: jal ra, FUNC
☐	0x00000014	0x00c000ef	jal x1, 0x0000000c	20: jal FUNC

实验步骤

1. 完成C程序到汇编程序，再到机器码的过程

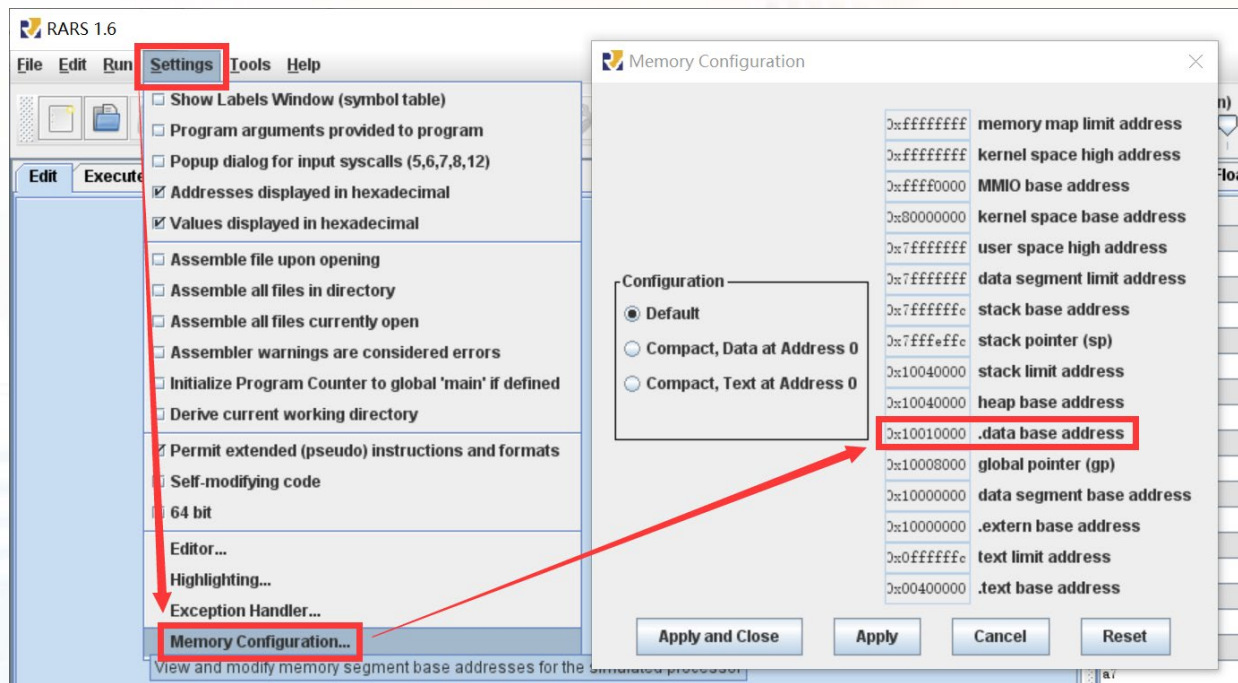
- ① 编写C程序
- ② 预编译，得到预编译文件 (.i)
- ③ 编译，得到汇编文件 (.s)，查看汇编代码
- ④ 汇编，得到目标文件 (.o)，反汇编并查看目标文件的机器码
- ⑤ 生成可执行文件，反汇编并查看可执行文件的机器码



实验步骤

2. 编写RV32汇编程序实现矩阵-向量乘法，并在RARS上调试、运行

① 打开RARS，查看存储配置模式



数据段基地址:
0x10010000

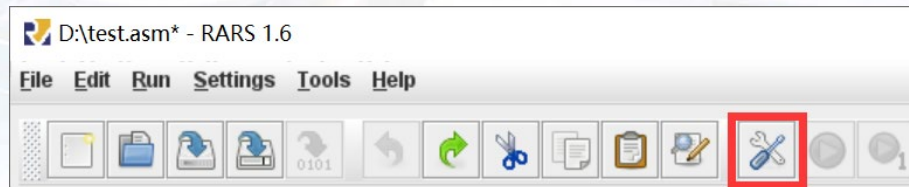
实验步骤

2. 编写RV32汇编程序实现浮点加法功能，并在RARS上调试、运行

② 编写RV32汇编程序

```
1 | .data                                ← 地址为0x10010000
2 |     Result_C: .space 16             # 4 x 32bit
3 |     Matrix_A: .byte 1, 2, 3, ...    # 4 x 8
4 |     ...                               ← 地址为0x10010010
5 | .text
6 |     ....
```

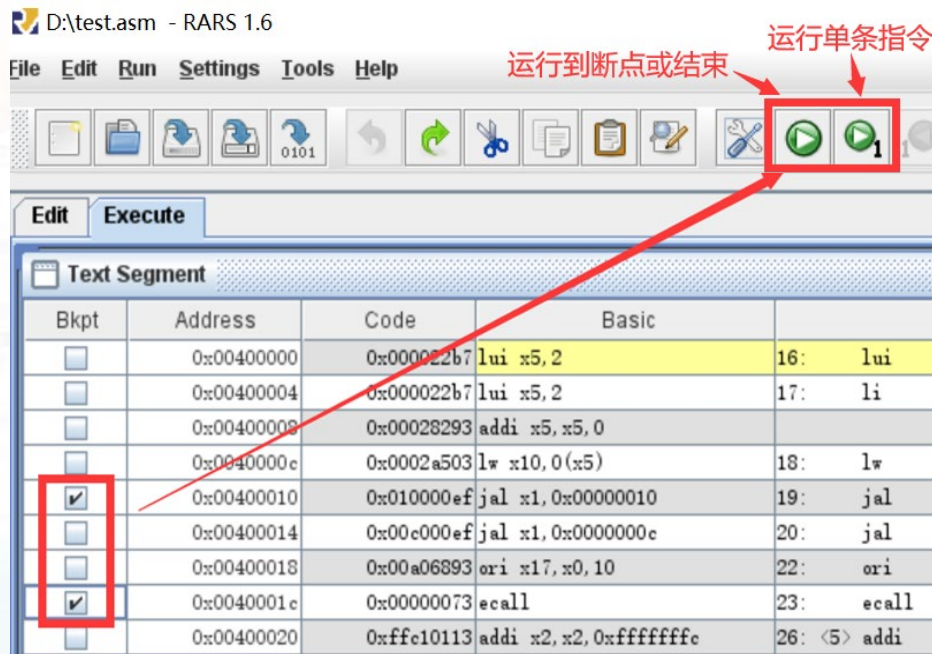
③ 对源代码进行汇编



实验步骤

2. 编写RV32汇编程序实现字符串匹配功能，并在RARS上调试、运行

④ 调试和运行



实验步骤

3. 借助大模型，把编写的RV32汇编程序以[内联汇编](#)的形式嵌入C程序，对比内联汇编与C语言实现有何不同
4. 用MAC4向量指令实现GEMV

```
mac4 rd, rs1, rs2    # rd += rs1[31:24]*rs2[31:24] +  
                      #          rs1[23:16]*rs2[23:16] +  
                      #          rs1[15: 8]*rs2[15: 8] +  
                      #          rs1[ 7: 0]*rs2[ 7: 0] +
```

5. 把所实现的GEMV集成到Llama2推理程序，下板测试推理性能

验收与提交

- **课上检查内容 (3分)**

- ① 在RARS中运行gemv_int8.asm汇编程序: 1分
- ② 在虚拟机运行gemv_int8_inline_asm.c程序: 1分
- ③ 在虚拟机运行gemv_int8_inline_asm_vec.c程序: 1分

- **提交内容 (6分)**

- ① 预编译文件(.i)、汇编文件(.s)、目标文件(.o)及其反汇编文件、可执行文件及其反汇编文件: 1分
- ② 课上检查的3个源文件: 1.5分
- ③ 实验报告 (按模板完成) : 3.5分

- 将上述文件打包成.zip, 以 “学号_姓名.zip” 命名提交到作业系统

- ◆ DDL: **5月17日 23:59** (以作业系统为准; 迟交会扣迟交分)
- ◆ 注意: **如有雷同, 双方均0分!**



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ